SOFTWARE DEFINED SECURITY · END-TERM PROJECT · PHASE 1–3

Explainable AI-Based SDN Intrusion Detection and Automated Flow Response

XAI-SDN-IDS: Ryu + Mininet/OVS · Random Forest · SHAP · Confidence-Gated Response Engine

Team: <names / roll numbers>
Faculty: Dr. Sumit Badotra · 15 October 2026

PHASE 1 · PROBLEM

Why SDN intrusion detection needs explanations and guard-rails

- SDN gives the controller a global, programmable view, so IDS/IPS can run as a controller app
- ML-based SDN IDSs report ≥ 99% accuracy, but:
  - Black box: operators cannot see why a host was flagged, so they cannot trust or audit it
  - Trigger-happy: most install a DROP on the first positive. A false positive becomes a self-inflicted DoS
  - Offline: evaluated on CSV datasets whose features a controller cannot actually measure
- Problem #15: collect OVS flow stats → ML IDS → feature-level explanations → install mitigation only when defined confidence/security conditions are satisfied

Objectives & scope

- O1 Multi-switch Mininet/OVS testbed with benign traffic + 4 attack classes
- O2 Ryu app: poll OpenFlow flow stats every 2 s → 14 per-source features
- O3 Train & compare multi-class detectors (Acc, P, R, F1, FPR, FNR)
- O4 SHAP feature-level explanations: local per alert, global per model
- O5 Confidence-Gated Response Engine: alert → meter rate-limit → self-expiring DROP
- O6 Evaluate detection/mitigation latency, victim QoS, controller overhead, gating ablation
- Scope: emulated SDN (OF1.3), volumetric + recon attacks from identifiable sources. Out of scope: payload/DPI, spoofed-source attribution

PHASE 1 · LITERATURE

Literature review: 16 verified papers (2020–2026)

Work	Approach	XAI	Auto-mitigation	Gating
[1] Elsayed 2020	InSDN dataset (ONOS testbed)	–	–	–
[2] Novaes 2020	LSTM + fuzzy, Floodlight	–	✓	statistical τ
[3] Tan 2020	trigger + KMeans/KNN	–	✓	2-stage filter
[5] Alashhab 2024	online ensemble, Ryu	–	✓	n/r
[6] El Kamel 2022	DNN rate limiting	–	✓	trust score
[7] Wang 2020	SHAP for IDS (NSL-KDD)	✓	–	–
[11] Saykat 2026	OptiXGB, InSDN, SHAP/LIME	✓	–	–
[12] Ahmad 2026*	XAI-SDN: RF + SHAP, Ryu	✓	offline	single τ=0.70
Ours	RF + SHAP + CGRE, live Ryu	✓	✓ graduated	G1–G4

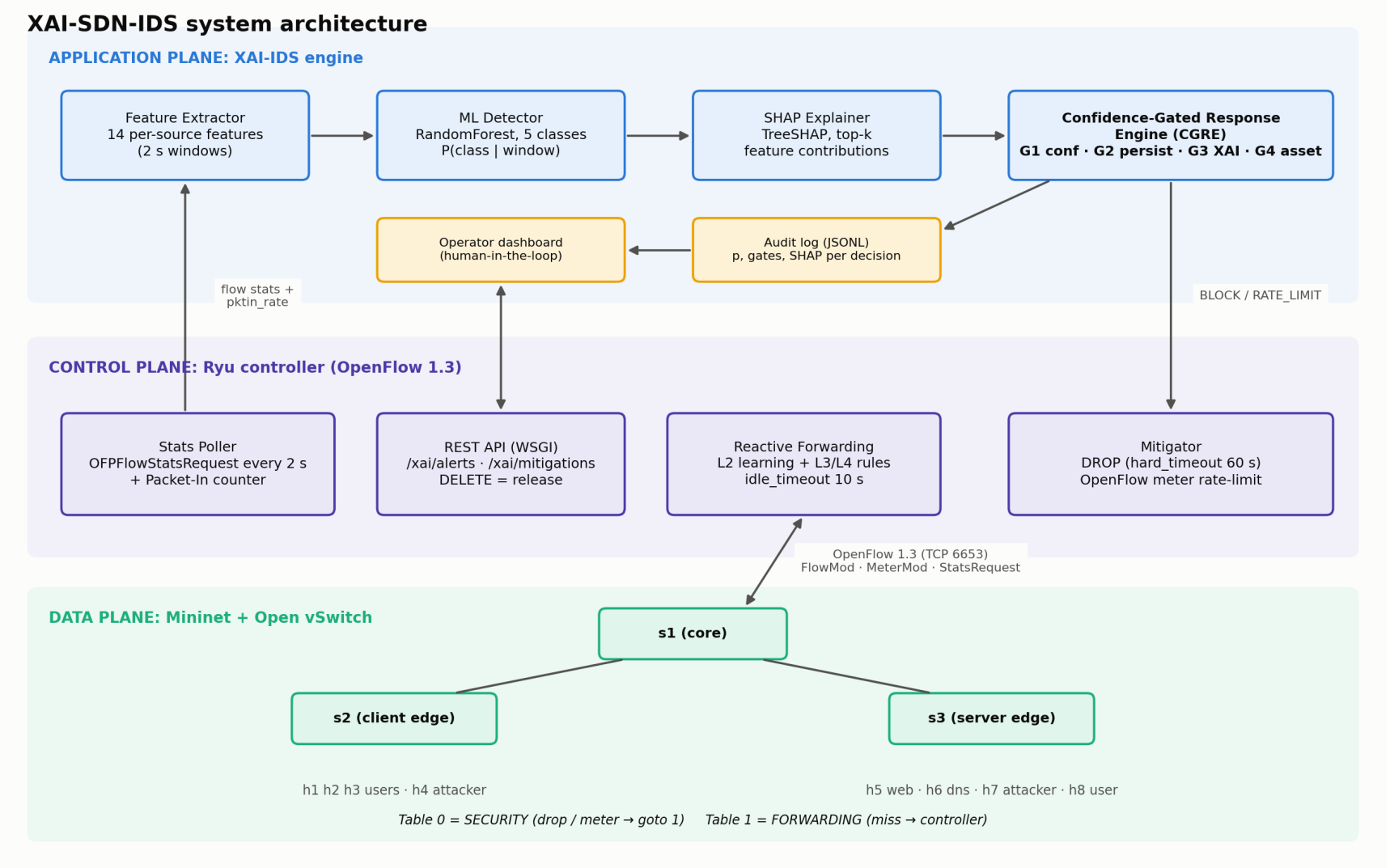
* preprint. Full table, synthesis and DOIs: docs/research/literature_review.md

Research gaps → our response

- RG1 Explanations never drive mitigation (offline [10][11] or analyst-only [12]) → SHAP is an input to the policy
- RG2 No multi-condition safety gate in SDN controllers → confidence + persistence + XAI + asset gates
- RG3 Explanation quality only scored offline [9][13] → per-alert runtime consistency check
- RG4 Graduated, self-expiring response is rare [6] → alert → meter → DROP with hard_timeout
- RG5 SDN XAI is mostly binary/offline → 5-class model on controller-native OpenFlow features
- RG6 Per-flow SHAP is ~31× slower [12] → explain only flagged per-source windows

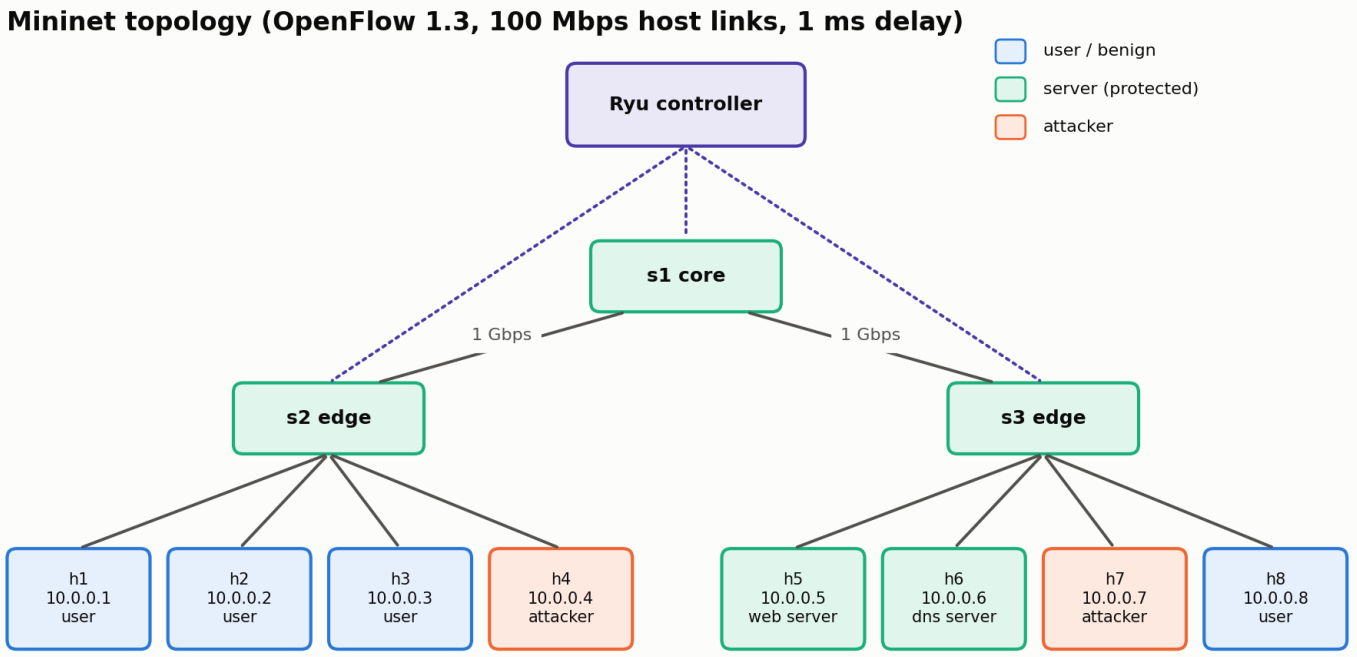
PHASE 2 · DESIGN

System architecture



PHASE 2 · DESIGN

Topology & OpenFlow pipeline



- Table 0 SECURITY: DROP / meter rules, default goto 1
- Table 1 FORWARDING: L2 learning + L3/L4 rules, miss → controller
- Match = (src, dst, proto, service port). Random source ports don't explode the flow table
- idle_timeout 10 s; stats polled every 2 s
- h5/h6 are protected assets (G4)

14 interpretable per-source features (2 s window)

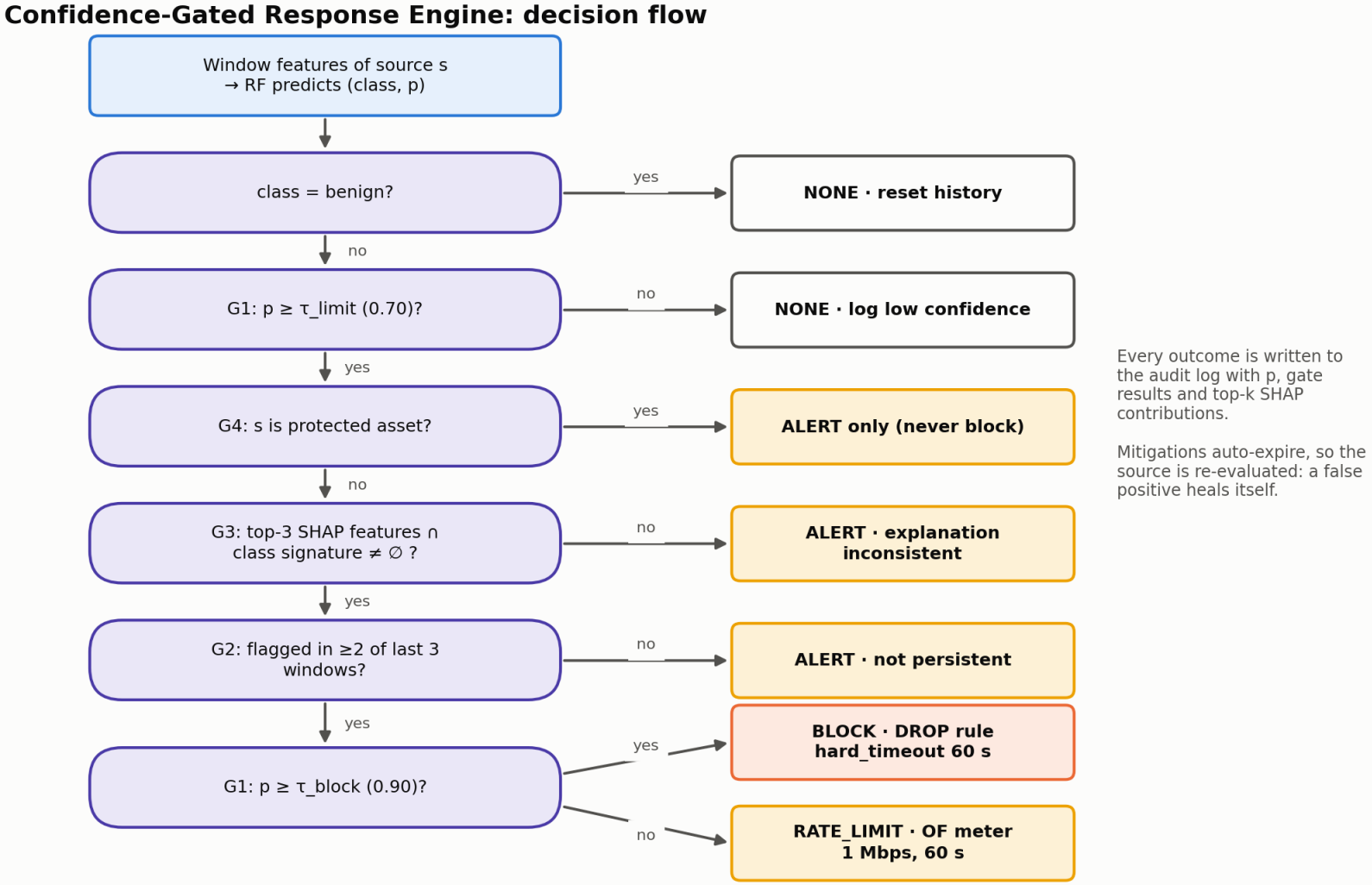
Feature(s)	Attack signal
pkt_rate, byte_rate	volume: floods
avg_pkt_size	tiny packets: SYN/ICMP flood
n_flows, new_flows	flow churn: scans
n_dst_ips, n_dst_ports	fan-out: sweeps/scans
frac_tcp / frac_udp / frac_icmp	protocol mix
pkts_per_flow	low: scan · high: flood/bulk
frac_small_flows	half-open / probe flows
mean_flow_duration	short vs persistent flows
pktin_rate	control-plane pressure

Classes:
benign
icmp_flood
syn_flood
udp_flood
port_scan

Benign includes heavy iperf3 transfers, so the model cannot learn 'rate = attack'.

PHASE 2 · SECURITY MECHANISM

Innovation: Confidence-Gated Response Engine



Every outcome is written to the audit log with p, gate results and top-k SHAP contributions.

Mitigations auto-expire, so the source is re-evaluated: a false positive heals itself.

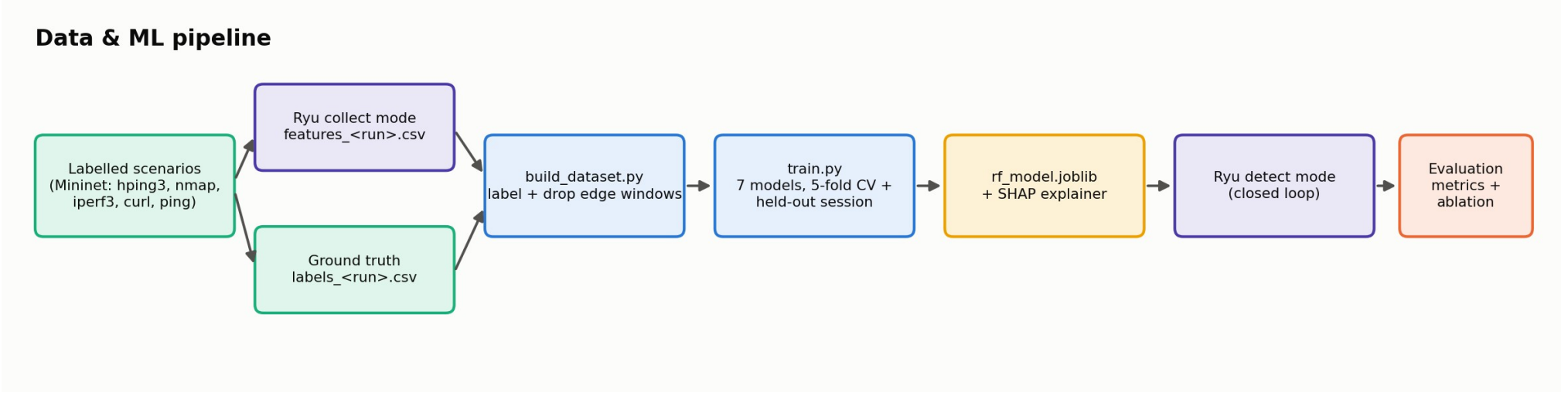
- G1 p ≥ 0.70 to act, ≥ 0.90 to block
- G2 ≥ 2 of last 3 windows
- G3 top-3 SHAP ∩ class signature
- G4 never block protected assets
- Rules auto-expire (60 s): a false positive heals itself
- Audit log + REST override (human-in-the-loop)

Technology choices

Choice	Why
Ryu 4.34 (Python)	ML/SHAP in-process; simple OF1.3 stats, meters, multi-table (ODL/ONOS: Java, heavy)
OpenFlow 1.3	meters for rate-limiting, multiple tables, per-flow counters
Mininet 2.3 + OVS 2.17	course standard, real OVS kernel datapath
Random Forest	best accuracy/latency trade-off on tabular data; exact fast TreeSHAP
SHAP TreeExplainer	additive, consistent, local + global; ms per alert
Docker Compose	reproducible on all laptops; Ryu needs Python 3.9

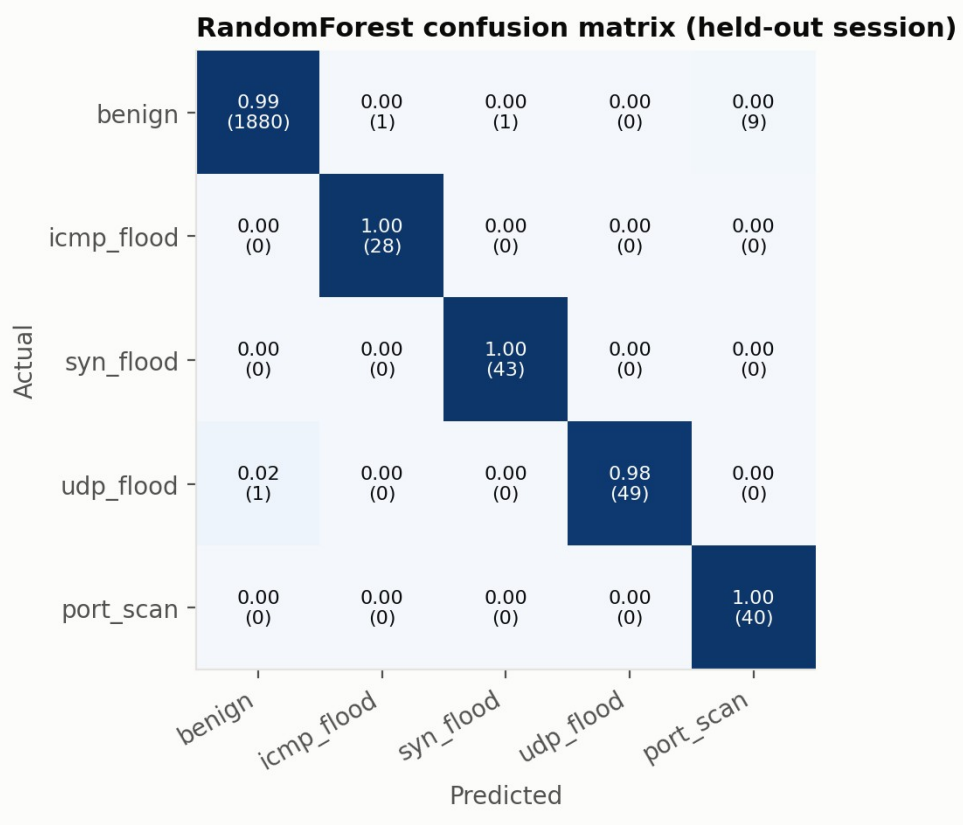
Implementation status

- Docker testbed: controller (Py3.9, Ryu, sklearn, SHAP) + privileged Mininet/OVS container
- controller/xai_ids_app.py: two-table pipeline, reactive forwarding, stats poller, mitigator (DROP + meters), REST API
- ids/: feature extractor, detector, SHAP explainer, response engine (policy.py), config
- traffic/scenarios.py: randomised labelled episodes (hping3 ICMP/SYN/UDP floods, nmap -sS/-sT, iperf3, curl, ping)
- ml/: dataset labelling, 7-model comparison, SHAP figures, audit/latency summary
- Unit tests for feature extraction and every CGRE gate (9/9 passing)
- Three modes: collect (dataset) · monitor (explain, never act) · detect (closed loop)



Preliminary results: held-out session

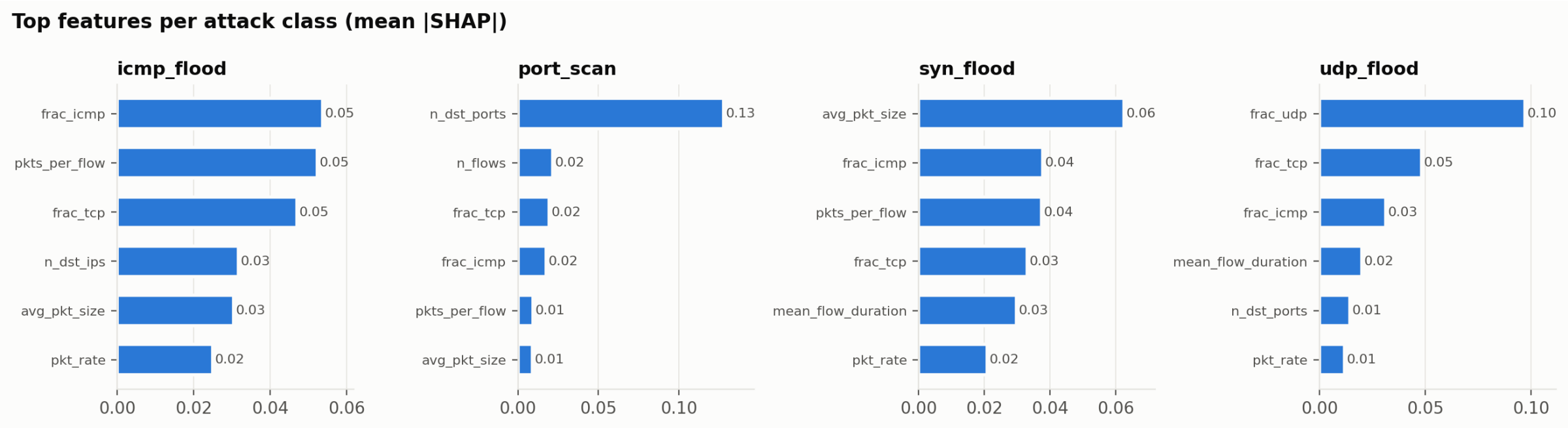
Model	Acc	Prec	Rec	F1	FPR	FNR	µs/ sample
RandomForest	0.994	0.952	0.995	0.971	0.006	0.006	37.2
ExtraTrees	0.989	0.914	0.994	0.950	0.011	0.006	34.7
HistGradientBoosting	0.984	0.868	0.993	0.922	0.016	0.006	33.7
DecisionTree	0.962	0.833	0.988	0.881	0.041	0.000	0.6
KNN	0.925	0.735	0.865	0.755	0.070	0.106	89.9
MLP	0.882	0.662	0.921	0.701	0.123	0.068	8.1
LogisticRegression	0.822	0.639	0.957	0.682	0.193	0.000	1.3



Trained on session 1, tested on an independent session 2 (different seed, rates, attackers, victims). FPR/FNR = benign-vs-attack view. Macro metrics over 5 classes.

PHASE 3 • XAI

Explanations: what drives each attack class



Real alert from the demo audit log:
10.0.0.4 → syn_flood p=0.95
because frac_tcp=1 (+0.18), avg_pkt_size=54 (+0.12), frac_icmp=0 (+0.11)
→ gates passed → BLOCK (auto-expires)

Per-class top features also validate the CGRE signatures (G3).

PHASE 3 · SECURITY RESULTS

Gating works: no innocent host blocked, every attack stopped

Policy	Innocent hosts impaired	Attacks mitigated	Mean time to mitigate
Naive (block on label)	2 (h6 server, h8 user)	12/12	5.1 s
G1 confidence	0	12/12	5.1 s
G1+G2 persistence	0	12/12	5.9 s
Full CGRE (G1-G4)	0	12/12	5.9 s

Replay of held-out session. Naive blocking hit flood victims whose replies looked like floods.

Live closed-loop demo	Flagged	Mitigated	Action · SHAP reason
Heavy benign 10 Mbps UDP (h1)	–	–	none (correct)
SYN flood h4 → h5	2.2 s	6.4 s	ALERT → BLOCK · frac_tcp=1, avg_pkt_size=54 B
Port scan h7 → h6	3.8 s	6.1 s	ALERT → BLOCK · n_dst_ports=538
UDP flood h4 → h8	2.3 s	2.4 s	RATE_LIMIT (p=.89) → BLOCK (p=.97) · frac_udp=1

Inference 9–31 ms, SHAP explanation ≈2.5 ms per alert. OVS meters verified (OF1.3, kbps, drop band).

What we learned: limitations that shape Phase 4

- Same-session CV overstates performance: CV F1 0.97–0.99 for all models, but 0.68–0.97 on an independent session
  - we always report held-out-session metrics
- Victim reflection: a host answering an ICMP flood emits an equal-rate ICMP stream
  - a 2,400 pps ICMP flood was detected with $p < 0.70$ (confidence gate held it back)
  - Phase 4: ICMP-type feature + reflection-aware gate G5
- Statistics lag up to 2 polling intervals → detection latency floor ≈ 2–4 s
- Labelling bug found and fixed: looping scans outlived their episode (now killed by process group)
- G2–G4 did not change outcomes in this session → adversarial/edge scenarios planned for Phase 5

ROADMAP

Plan for Phases 4–7 (final evaluation 16–19 Nov)

When	Work
16–22 Oct	More sessions & intensities (incl. low-rate), tuning, InSDN cross-check
23–29 Oct	Closed-loop CGRE hardening, meters, audit log, REST API
30 Oct–5 Nov	Experiments: detection/mitigation latency, throughput/RTT/loss, CPU/mem, ablation naive vs gated, SHAP vs LIME
6–12 Nov	Operator dashboard, final report & figures
13–15 Nov	Demo rehearsal, viva preparation, backup recording

Thank you. Questions?

- Code: xai-sdn-ids/ (README → quick start)
- Key references: [1] InSDN (IEEE Access 2020) · [6] El Kamel (Comp. Comm. 2022) · [7] Wang (IEEE Access 2020) · [12] XAI-SDN (arXiv 2026) · [13] Alquliti (arXiv 2025)
- Full IEEE-style reference list in the Phase 1–3 report